



Required RailBlock AI solution

RailBlock AI should be built as a **human-approved, explainable hybrid AI decision-support system** for integrated railway maintenance block planning.

It should combine:

```
TMS maintenance and defects
+ SMMS signalling maintenance
+ TDMS traction/OHE maintenance
+ COA corridor and asset availability
+ train timetable
+ goods-train forecast
+ crew, machine, material, and resource availability
+ historical execution results
```

The system should produce:

```
prioritized maintenance tasks
+ feasible candidate windows
+ integrated departmental blocks
+ weekly and monthly plans
+ explanations
+ approval workflow
+ department-wise visibility
+ execution tracking
+ plan revisions
+ baseline-versus-optimized metrics
```

Indian Railways' rolling-block guidance emphasizes advance block planning, integration of civil, electrical, S&T, and project work, and avoiding separate blocks in the same section when work can be coordinated. Railway maintenance scheduling research also supports combining work, crew, possession, and cost considerations in one planning model. ^[1] ^[2] ^[3]

System boundary

In scope

```
Importing synthetic or authorized source data
Data validation and normalization
Asset and corridor mapping
Task prioritization
Timetable conflict checking
```

Goods-forecast risk analysis
Resource feasibility checking
Multi-department task grouping
Weekly plan generation
Monthly plan generation
CP-SAT optimization
Greedy fallback scheduling
Independent plan validation
Officer editing
Approval and publication
Department-wise plan visibility
Execution recording
Performance metrics
Versioned replanning
Audit history
CSV/PDF export

Out of scope

Autonomous railway control
Automatic signal operation
Automatic power isolation
Live infrastructure commands
Train dispatching
Final safety authorization
Unsupervised plan publication
Real railway deployment without authorization

The application must be described as:

A human-approved planning and decision-support prototype.

It must not be described as:

An autonomous railway-control system.
A railway-certified safety system.
A live railway-integrated system unless authorized interfaces exist.

High-level architecture

```
graph TD
    Users --> Dashboard[React web dashboard]
    Dashboard --> API[REST API]
    API --> Orchestrator[Planning orchestrator]
    Orchestrator --- Auth[Authentication and role context]
    Orchestrator --- Import[Import service]
    Orchestrator --- Validation[Validation service]
```

Users
↓
React web dashboard
↓
REST API
↓
Planning orchestrator
├─ Authentication and role context
├─ Import service
└─ Validation service

- |— Normalization service
- |— Asset and corridor mapping
- |— Priority engine
- |— Risk engine
- |— Candidate-window engine
- |— Compatibility and grouping engine
- |— CP-SAT optimization engine
- |— Greedy fallback scheduler
- |— Independent plan validator
- |— Approval and publication service
- |— Department visibility service
- |— Execution service
- |— Metrics service
- |— Revision and replan service
- |— Notification service
- |— Export service
- ↓
- Canonical database
 - |— SQLite for local prototype
 - |— PostgreSQL option for multi-user deployment

Complete component architecture

1. Frontend layer
 - React 18
 - Vite
 - React Router
 - Dashboard
 - Data import
 - Task inbox
 - Planner
 - Execution
 - Metrics
 - Department views
 - Notifications
 - Secondary detail pages
2. API layer
 - FastAPI
 - Pydantic request and response schemas
 - OpenAPI documentation
 - Role and department context
 - Structured errors
 - Audit-aware endpoints
3. Domain layer
 - Task priority
 - Timetable conflict logic
 - Goods forecast risk
 - Resource compatibility
 - Multi-department grouping
 - Candidate-window generation
 - Plan generation
 - Plan validation
 - Plan editing

- Replanning
- Execution feedback
- 4. Data integration layer
 - TMS adapter
 - SMMS adapter
 - TDMS adapter
 - COA adapter
 - Timetable adapter
 - Goods forecast adapter
 - Resource adapter
- 5. Persistence layer
 - SQLAlchemy ORM
 - SQLite WAL for local mode
 - PostgreSQL-compatible schema
 - Transactions
 - Foreign keys
 - Indexes
 - Versioned plans
 - Audit records
- 6. Optimization layer
 - Baseline FCFS scheduler
 - CP-SAT optimizer
 - Deterministic greedy fallback
 - Independent validator
 - Objective breakdown
- 7. Operations layer
 - Docker Compose
 - Nginx
 - One startup script
 - Health checks
 - CI
 - Test suite
 - Documentation

Canonical domain entities

Use the following entities without combining unrelated concepts.

Department
UserContext
Corridor
Section
Line
Asset
Task
TaskDependency
TrainMovement
GoodsForecast
Resource
ResourceAvailability
CandidateWindow

TaskWindowCandidate
TaskGroup
BlockPlan
Block
BlockTask
PlanRevision
PlanChange
Approval
ExecutionRecord
Notification
RuleConfiguration
ImportRun
AuditEvent

Entity meanings

Corridor:

A railway route or operational corridor.

Section:

A subsection of a corridor.

Line:

UP, DOWN, single line, loop, or another explicitly defined line identity.

Asset:

A physical or logical infrastructure asset.

Task:

A maintenance, inspection, repair, renewal, or project activity.

CandidateWindow:

A possible period during which work may be scheduled.

TaskWindowCandidate:

A task-window combination that passed preliminary feasibility checks.

TaskGroup:

A compatible collection of tasks.

Block:

A selected operational possession or work block.

BlockTask:

The association between a Block and a Task.

BlockPlan:

A collection of Blocks for a planning horizon.

PlanRevision:

A new version derived from an earlier plan.

Approval:

A record of review or approval action.

ExecutionRecord:

The actual result of a selected block or block task.

Notification:

A department-facing event about a plan, revision, assignment, or execution change.

Required database relationships

Corridor 1 → many Sections
Section 1 → many Lines
Corridor 1 → many Assets
Asset 1 → many Tasks
Corridor 1 → many Tasks
Corridor 1 → many TrainMovements
Corridor 1 → many GoodsForecasts
Task many ↔ many Resources
Task many ↔ many Tasks through dependencies
BlockPlan 1 → many Blocks
Block 1 → many BlockTasks
BlockTask → one Task
PlanRevision → one base BlockPlan
PlanChange → one PlanRevision
Block → zero or many Approval records
Block or BlockTask → zero or many ExecutionRecords
Department → many UserContexts
Department → many Notifications

Identifier rules

COR-*	Corridor ID
SEC-*	Section ID
LIN-*	Line ID
AST-*	Asset ID
TSK-*	Task ID
TRN-*	Train ID
RES-*	Resource ID
WND-*	Candidate window ID
GRP-*	Task-group ID
BLK-*	Selected block ID
PLAN-*	Plan ID
REV-*	Revision ID
EXE-*	Execution record ID

Never use a **WND-*** candidate-window identifier where a selected **BLK-*** identifier is required.

Data-ingestion architecture

The ingestion pipeline must be:

Source file or API

↓

```
Source adapter
↓
Raw records
↓
Structural validation
↓
Header and field normalization
↓
Canonical transformation
↓
Referential validation
↓
Operational validation
↓
Duplicate detection
↓
Preview
↓
User confirmation
↓
Transactional persistence
↓
Import summary and audit record
```

Source adapters

```
TMSAdapter
SMMSAdapter
TDMSAdapter
COAAdapter
TimetableAdapter
GoodsForecastAdapter
ResourceAdapter
```

Every adapter should provide:

```
source_name
expected_columns
load()
normalize()
validate()
to_canonical()
```

Import result

Every import must return:

```
import_run_id
source_name
received_count
accepted_count
rejected_count
```

```
warning_count
errors
warnings
duplicate_count
started_at
completed_at
```

Each error must include:

```
row
field
severity
code
message
```

Example:

```
{
  "row": 18,
  "field": "corridor_id",
  "severity": "ERROR",
  "code": "UNKNOWN_CORRIDOR",
  "message": "Corridor COR-99 does not exist."
}
```

Canonical task structure

Each task should include:

```
task_id
source_system
department
asset_id
corridor_id
section_id
line_id
location_from_km
location_to_km
task_type
description
severity
safety_score
urgency_score
asset_criticality
operational_impact
overdue_days
coordination_value
resource_readiness
estimated_duration_minutes
setup_duration_minutes
required_block_type
requires_traffic_block
```



```
requires_power_isolation
requires_signal_disconnection
earliest_start
deadline
dependency_task_ids
required_resource_ids
status
priority_score
priority_rank
priority_reason
```

Priority architecture

The priority engine is an explainable feature-based component.

Use:

```
P =
0.30S
+ 0.20U
+ 0.20C
+ 0.15O
+ 0.10D
+ 0.05R
```

Where:

```
S = safety criticality
U = urgency or overdue status
C = asset criticality
O = operational impact
D = coordination value
R = resource readiness
```

Every factor must be normalized to 0–100.

The output must contain:

```
priority_score
priority_rank
priority_band
factor_values
factor_weights
priority_breakdown
priority_reason
rule_configuration_version
```

Example explanation:

Priority 91.4:

- high safety criticality;
- 12 overdue days;
- critical asset;
- significant operational impact;
- compatible with an existing corridor block;
- required resources available.

Priority must never override hard constraints.

Candidate-window architecture

A candidate window is generated from:

```
planning horizon
corridor availability
section and line
timetable gaps
goods forecast
maintenance calendar
minimum buffer
maximum block duration
required isolation condition
resource availability
```

Each candidate window should contain:

```
window_id
service_date
corridor_id
section_id
line_id
start_time
end_time
available_minutes
block_type
requires_power_isolation
requires_signal_disconnection
expected_train_count
goods_risk_score
risk_band
availability_source
rejection_reason
status
```

Use one time standard:

```
Timezone: Asia/Kolkata
Date: YYYY-MM-DD
Optimization time: integer minutes from midnight
```

A prototype may use configurable templates such as:

```
01:00-03:00  
13:30-15:30  
02:00-06:00
```

These must be labelled as:

```
Synthetic prototype windows, not official railway availability.
```

Timetable and forecast analysis

For each task-window pair, check:

```
passenger train overlap  
goods train overlap  
forecast confidence  
corridor  
section  
line  
buffer  
block type  
power state  
signalling state  
resource availability
```

Classify results as:

```
HARD_CONFLICT  
SOFT_RISK  
FEASIBLE
```

Example:

```
Passenger train overlapping protected interval:  
HARD_CONFLICT
```

```
Goods forecast with low confidence:  
SOFT_RISK
```

```
Goods forecast with configured high confidence:  
HARD_CONFLICT or configurable severe penalty
```

Goods forecast should normally affect risk or ranking unless an authorized operational rule makes it a hard restriction.

Compatibility and grouping

Two tasks may be grouped only if all applicable checks pass:

```
same corridor
compatible section
compatible line
compatible date
compatible block type
compatible power requirements
compatible signalling requirements
compatible access requirements
compatible resources
compatible dependencies
combined duration ≤ maximum block duration
```

The result must provide reasons:

```
{
  "compatible": false,
  "reasons": [
    "Task requires power isolation.",
    "Second task is traffic-only.",
    "Combined duration exceeds 240 minutes."
  ]
}
```

Do not group tasks solely because they share a corridor.

Optimization architecture

Use two planners:

```
Baseline FCFS scheduler
Optimized CP-SAT scheduler
```

Baseline

The FCFS baseline should:

```
sort tasks by priority or arrival order
assign the first feasible window
avoid hard conflicts
avoid resource conflicts
create no intentional integrated groups
```

CP-SAT planner

The optimization flow should be:

```
All tasks
↓
Candidate-window prefilter
↓
Task-window candidates
↓
Compatibility grouping
↓
CP-SAT model
↓
Solver result
↓
Independent validator
↓
Draft BlockPlan
```

CP-SAT is suitable for interval scheduling and no-overlap constraints, including fixed and optional intervals. ^[4]
^[5]

Hard constraints

The solver must not violate:

```
task assigned at most once
task duration fits
protected train intervals
corridor compatibility
section compatibility
line compatibility
block type compatibility
power isolation compatibility
signalling disconnection compatibility
resource non-overlap
machine non-overlap
dependency ordering
maximum block duration
deadline rules
approved-block preservation
completed-task preservation
```

Soft objectives

Use a transparent objective such as:

```
maximize:
  completed priority value
+ critical-task benefit
+ overdue-risk reduction
```

```
+ integrated-group benefit
+ asset-availability benefit
- train-risk penalty
- unused-block-time penalty
- resource travel/setup cost
- lateness penalty
- disruption penalty
```

The plan response must expose objective components.

Do not expose only one unexplained score.

Solver results

Valid statuses:

```
OPTIMAL
FEASIBLE
INFEASIBLE
UNKNOWN
TIME_LIMIT
FALLBACK_USED
VALIDATION_FAILED
```

A solver status of `OPTIMAL` means optimal only relative to the encoded prototype model. It does not mean operationally approved.

Independent plan validation

Every solver result must pass a second validator.

Validation order:

```
A. Plan structure
B. Task uniqueness
C. Task-window fit
D. Train conflict
E. Goods-risk rule
F. Resource conflict
G. Line and corridor compatibility
H. Power and signalling compatibility
I. Dependency order
J. Locked and completed preservation
K. Duration and buffer rules
L. Approval-state rules
```

If validation fails:

```
do not save as approved
do not publish
```

```
do not show success
return violation details
```

Plan lifecycle

Plan states

```
DRAFT
UNDER_REVIEW
APPROVED
PUBLISHED
REVISION_REQUESTED
SUPERSEDED
REJECTED
```

Block states

```
GENERATED
UNDER_REVIEW
APPROVED
PUBLISHED
IN_PROGRESS
COMPLETED
PARTIALLY_COMPLETED
CANCELLED
REJECTED
```

Task states

```
PENDING
VALIDATION_FAILED
ELIGIBLE
SCHEDULED
LOCKED
IN_PROGRESS
COMPLETED
PARTIALLY_COMPLETED
DEFERRED
CANCELLED
```

Only valid transitions should be allowed.

Editing and versioning

Draft plan

A draft may be edited if:

```
user has planner permission
plan is still editable
edit passes validation
edit reason is recorded
```

Approved plan

An approved or published plan must not be directly edited.

Use:

```
approved plan
↓
create revision
↓
copy plan structure
↓
edit revision
↓
validate revision
↓
submit for review
↓
approve revision
↓
publish revision
↓
supersede previous plan
```

Supported edits

```
move task to another feasible window
add eligible task
remove task with reason
change task sequence
split compatible group
merge compatible tasks
change assigned resource
edit operational notes
```

Prohibited edits

```
move completed task
delete execution record
create train conflict
create resource conflict
```



```
violate power isolation
violate signalling requirements
exceed block duration
break dependencies
silently remove approved work
```

Department architecture

Use prototype roles:

```
CONTROL_OFFICE
ENGINEERING
S_AND_T
TRACTION
PROJECTS
VIEWER
ADMIN
```

Department view

Every department should see:

```
its own tasks
its assigned blocks
integrated tasks from other departments
corridor and line
block type
start and end time
power/signalling requirements
approval status
execution status
plan version
change history
reasons for displacement or deferment
```

Other departments' work should remain visible as coordination context. Do not expose unrelated confidential data.

Department workflow

```
department logs in or selects demo department
↓
views relevant approved plans
↓
sees own tasks prominently
↓
sees integrated tasks from other departments
↓
reviews block conditions
↓
receives revision notifications
```

↓
tracks execution status

Approval architecture

Approval must be backend-controlled.

Before approval, check:

```
plan exists
plan is in reviewable state
plan passed independent validation
all hard violations are resolved
required fields exist
approver identity exists
approver has permission
```

On approval, create:

```
Approval record
AuditEvent
plan status change
department notifications
```

Approval must not be implemented only as a frontend button.

Execution architecture

The canonical endpoint should be:

```
POST /api/blocks/{block_id}/execution
```

The execution request should contain:

```
actual_start
actual_end
status
completed_task_ids
partially_completed_task_ids
cancelled_task_ids
reason
asset_status
train_impact
notes
recorded_by
```

Validation:

```
actual_end ≥ actual_start
completed task belongs to block
cancelled status requires reason
deferred status requires reason
partial completion requires task selection or notes
duplicate submission returns 409 or documented idempotent result
```

Execution must update:

```
Block status
BlockTask status
Task status
ExecutionRecord
metrics
audit history
```

Execution history must survive replanning.

Replanning architecture

Replanning is triggered by:

```
emergency task
train timetable change
goods forecast change
resource unavailability
block cancellation
partial completion
new critical defect
```

Replanning flow:

```
current plan
↓
identify locked blocks
↓
identify approved blocks
↓
identify completed tasks
↓
freeze protected work
↓
add new or changed data
↓
regenerate candidate windows
↓
recalculate priorities and risks
↓
rebuild compatible groups
↓
run optimizer
```

```
↓
validate
↓
create new plan version
↓
show preserved, moved, displaced, and new tasks
↓
submit for review
```

The response must include:

```
base_plan_id
new_plan_id
preserved_blocks
preserved_tasks
moved_tasks
displaced_tasks
new_tasks
displacement_reasons
validation_result
```

Complete unambiguous workflow

```
START
↓
Select planning horizon
↓
Select weekly or monthly mode
↓
Select corridors, sections, lines, or all
↓
Load TMS data
↓
Load SMMS data
↓
Load TDMS data
↓
Load COA corridor and asset data
↓
Load timetable data
↓
Load goods forecast data
↓
Load resource availability data
↓
Run structural validation
↓
Are required fields and data types valid?
├─ NO → create row-level errors → show rejected records → STOP IMPORT
└─ YES
↓
Normalize IDs, departments, locations, dates, times, and units
↓
```

```
Run referential validation
↓
Do assets, corridors, lines, resources, and dependencies exist?
├─ NO → create referential errors → reject affected records → STOP PLANNING
└─ YES
↓
Run operational validation
↓
Are durations, deadlines, block types, isolation states, and dependencies valid?
├─ NO → create operational errors → reject affected records → STOP PLANNING
└─ YES
↓
Detect duplicates
↓
Persist accepted records transactionally
↓
Create import-run summary and audit event
↓
Map tasks to corridor, section, line, asset, and location
↓
Extract task features
↓
Calculate safety score
↓
Calculate urgency score
↓
Calculate asset-criticality score
↓
Calculate operational-impact score
↓
Calculate coordination-value score
↓
Calculate resource-readiness score
↓
Calculate explainable priority score and rank
↓
Generate timetable-aware candidate windows
↓
Add configured safety buffers
↓
Check passenger-train intervals
↓
Check goods-train forecast risk
↓
Check corridor, section, and line availability
↓
Check block and isolation compatibility
↓
Check crew, machine, and material availability
↓
Are candidate windows feasible?
├─ NO → mark window rejected with reason
└─ YES → store candidate window
↓
Create feasible task-window pairs
↓
```

```
Find compatible task groups
↓
Check spatial compatibility
↓
Check operational compatibility
↓
Check resource compatibility
↓
Check combined duration
↓
Create baseline FCFS plan
↓
Create CP-SAT optimization model
↓
Apply hard constraints
↓
Apply soft objective weights
↓
Run CP-SAT within configured time limit
↓
Did CP-SAT return a usable solution?
├─ NO → run deterministic greedy fallback
└─ YES → use feasible CP-SAT result
↓
Run independent plan validator
↓
Does the generated plan violate any hard constraint?
├─ YES → reject result → display violations → do not save as valid plan
└─ NO
↓
Calculate baseline-versus-optimized metrics
↓
Create DRAFT BlockPlan
↓
Display:
- blocks
- tasks
- departments
- train checks
- resource checks
- risk
- priority explanations
- solver status
- unscheduled reasons
↓
Planner reviews plan
↓
Does planner edit the draft?
├─ YES → apply edit → validate edit → save draft if valid
└─ NO
↓
Does planner reject the draft?
├─ YES → record reason → mark rejected → END
└─ NO
↓
Submit plan for review
```

```

    ↓
Authorized officer reviews
    ↓
Does officer approve?
  ├── NO → record rejection reason → return to draft or rejected state
  └── YES
    ↓
Create approval record and audit event
    ↓
Publish plan
    ↓
Generate department-specific approved views
    ↓
Notify Engineering, S&T, Traction, Projects, and Control Office
    ↓
Departments view assigned and integrated work
    ↓
Execute published block
    ↓
Record actual start, actual end, completion, cancellation, delay, and impact
    ↓
Update block, task, asset, and execution states
    ↓
Calculate planned-versus-actual metrics
    ↓
Did an emergency, cancellation, new forecast, or timetable change occur?
  ├── NO → continue monitoring
  └── YES
    ↓
Freeze completed and approved/locked work
    ↓
Create new plan revision
    ↓
Recalculate windows, risks, priorities, and groups
    ↓
Optimize remaining unlocked work
    ↓
Validate new revision
    ↓
Display changes and displaced-task reasons
    ↓
Review and approve revision
    ↓
Publish new version
    ↓
END or continue rolling planning cycle

```

Compact flowchart text

```

START
→ Select horizon
→ Import six/seven data domains
→ Structural validation
→ Normalize data
→ Referential validation

```

- Operational validation
- Persist accepted data
- Map tasks to corridor/section/line/assets
- Extract features
- Calculate priority and explanations
- Generate candidate windows
- Check timetable
- Check goods forecast
- Check buffers and isolation
- Check resources
- Reject infeasible windows
- Generate task-window pairs
- Group compatible tasks
- Build FCFS baseline
- Run CP-SAT
- If timeout/infeasible, run fallback
- Independent validation
- Calculate metrics
- Create draft plan
- Planner edits or submits
- Officer approves or rejects
- Publish approved plan
- Show department-specific views
- Execute block
- Record actuals
- Update metrics
- Detect change or emergency
- Freeze completed/approved work
- Create revision
- Re-optimize unlocked work
- Validate revision
- Review and publish
- Repeat rolling cycle

API structure

```
GET /health

GET /api/departments
GET /api/corridors
GET /api/assets
GET /api/tasks
GET /api/trains
GET /api/resources

POST /api/import/tasks
POST /api/import/corridors
POST /api/import/assets
POST /api/import/trains
POST /api/import/goods-forecast
POST /api/import/resources
GET /api/import/summary
GET /api/import/{import_run_id}

GET /api/windows
```



```
GET /api/windows/{window_id}

POST /api/plans/generate
GET /api/plans
GET /api/plans/{plan_id}
POST /api/plans/{plan_id}/validate
POST /api/plans/{plan_id}/approve
POST /api/plans/{plan_id}/reject
POST /api/plans/{plan_id}/replan
GET /api/plans/{plan_id}/history
GET /api/plans/{plan_id}/changes
GET /api/plans/{plan_id}/export

POST /api/plans/{plan_id}/revisions
PATCH /api/plans/{plan_id}/draft-blocks/{block_id}
POST /api/plans/{plan_id}/submit-review

GET /api/approved-plans
GET /api/plans/{plan_id}/department-view
GET /api/notifications

POST /api/blocks/{block_id}/execution
GET /api/execution/plan/{plan_id}
GET /api/execution
POST /api/execution/{id}                deprecated compatibility alias

POST /api/conflicts/detect
POST /api/optimize
GET /api/metrics
GET /api/metrics/{plan_id}

GET /api/compatibility/priority-weights
GET /api/compatibility/optimizer-weights
GET /api/compatibility/hard-constraints
GET /api/compatibility/ai-model
GET /api/tasks/{task_id}/priority-explanation
GET /api/plans/{plan_id}/explanations
```

Frontend structure

Primary navigation:

1. Dashboard
2. Import and validate
3. Prioritize tasks
4. Plan, review, and approve
5. Execute and measure

Dashboard

live record counts
data freshness
critical overdue tasks
latest approved plan
upcoming blocks
department alerts
baseline-versus-optimized summary
workflow stepper
synthetic-data warning

Import page

source selector
drag-and-drop CSV
validation preview
accepted records
rejected records
row-level errors
warnings
import summary
last import timestamp

Task inbox

priority score
rank
priority band
breakdown
reason
department filter
corridor filter
severity filter
status filter
compatibility checker

Planner

weekly/monthly switch
corridor selector
candidate windows
generated blocks
approved blocks
executed blocks
train overlay
goods-risk overlay
resource panel
validation panel
solver status

objective breakdown
edit draft
create revision
approve
reject
publish

Department view

department selector
my tasks
integrated tasks
approved blocks
upcoming work
change notifications
execution state
plan version
approval history

Execution

plan selector
block selector
planned values
actual values
completed tasks
partial tasks
cancellation reason
asset status
train impact
notes
submit result
replan action

Metrics

FCFS baseline
optimized plan
blocks
block minutes
scheduled tasks
critical tasks
integrated groups
conflicts
unused time
resource utilization
planned-versus-actual duration

Folder structure

```
PROJ/
├── backend/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── alembic.ini
│   ├── migrations/
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database.py
│       ├── dependencies.py
│       ├── models/
│       │   ├── department.py
│       │   ├── user_context.py
│       │   ├── corridor.py
│       │   ├── section.py
│       │   ├── line.py
│       │   ├── asset.py
│       │   ├── task.py
│       │   ├── task_dependency.py
│       │   ├── train.py
│       │   ├── goods_forecast.py
│       │   ├── resource.py
│       │   ├── resource_availability.py
│       │   ├── candidate_window.py
│       │   ├── task_window_candidate.py
│       │   ├── task_group.py
│       │   ├── plan.py
│       │   ├── block.py
│       │   ├── block_task.py
│       │   ├── plan_revision.py
│       │   ├── plan_change.py
│       │   ├── approval.py
│       │   ├── execution_record.py
│       │   ├── notification.py
│       │   ├── rule_configuration.py
│       │   ├── import_run.py
│       │   └── audit_event.py
│       ├── schemas/
│       │   ├── common.py
│       │   ├── department.py
│       │   ├── task.py
│       │   ├── train.py
│       │   ├── corridor.py
│       │   ├── asset.py
│       │   ├── window.py
│       │   ├── group.py
│       │   ├── block.py
│       │   ├── plan.py
│       │   ├── revision.py
│       │   ├── execution.py
│       │   ├── notification.py
│       │   └── validation.py
```

```
├── adapters/
│   ├── base.py
│   ├── tms.py
│   ├── smms.py
│   ├── tdms.py
│   ├── coa.py
│   ├── timetable.py
│   ├── goods_forecast.py
│   └── resources.py
├── services/
│   ├── ingestion.py
│   ├── normalization.py
│   ├── validation.py
│   ├── mapping.py
│   ├── priority.py
│   ├── risk.py
│   ├── candidate_windows.py
│   ├── compatibility.py
│   ├── grouping.py
│   ├── baseline.py
│   ├── optimizer.py
│   ├── fallback.py
│   ├── plan_validator.py
│   ├── state_machine.py
│   ├── approvals.py
│   ├── revisions.py
│   ├── visibility.py
│   ├── notifications.py
│   ├── execution.py
│   ├── replanning.py
│   ├── metrics.py
│   └── export.py
├── routers/
│   ├── health.py
│   ├── imports.py
│   ├── tasks.py
│   ├── corridors.py
│   ├── trains.py
│   ├── windows.py
│   ├── plans.py
│   ├── revisions.py
│   ├── approvals.py
│   ├── departments.py
│   ├── notifications.py
│   ├── execution.py
│   ├── conflicts.py
│   ├── optimizer.py
│   ├── metrics.py
│   └── export.py
└── tests/
    ├── test_health.py
    ├── test_ingestion.py
    ├── test_validation.py
    ├── test_priority.py
    ├── test_windows.py
    └── test_conflicts.py
```

```
├── test_grouping.py
├── test_optimizer.py
├── test_plan_validator.py
├── test_approval.py
├── test_visibility.py
├── test_revisions.py
├── test_execution.py
├── test_replanning.py
├── test_metrics.py
├── test_end_to_end.py
├── frontend/
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── package.json
│   ├── vite.config.ts
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── types/
│       │   └── index.ts
│       ├── services/
│       │   ├── api.ts
│       │   ├── errors.ts
│       │   └── formatters.ts
│       ├── components/
│       │   ├── Navbar.tsx
│       │   ├── Card.tsx
│       │   ├── Gantt.tsx
│       │   ├── PlanStatus.tsx
│       │   ├── ValidationPanel.tsx
│       │   ├── DepartmentSelector.tsx
│       │   ├── ChangeHistory.tsx
│       │   ├── NotificationPanel.tsx
│       │   └── MetricsChart.tsx
│       └── pages/
│           ├── Dashboard.tsx
│           ├── DataImport.tsx
│           ├── TaskInbox.tsx
│           ├── Planner.tsx
│           ├── DepartmentPlans.tsx
│           ├── Execution.tsx
│           ├── Metrics.tsx
│           ├── Corridors.tsx
│           ├── Trains.tsx
│           ├── Conflicts.tsx
│           └── Optimizer.tsx
├── data/
│   ├── generate_sample.py
│   └── sample/
│       ├── corridors.csv
│       ├── sections.csv
│       ├── lines.csv
│       ├── assets.csv
│       ├── tasks.csv
│       ├── trains.csv
│       └── goods_forecast.csv
```

```
|
|   ├── resources.csv
|   └── resource_availability.csv
├── docs/
|   ├── architecture.md
|   ├── data-dictionary.md
|   ├── problem-understanding.md
|   ├── implementation-spec.md
|   ├── workflow.txt
|   ├── demo-script.md
|   ├── demo-evidence.md
|   ├── manual-acceptance.md
|   ├── final-release-audit.md
|   └── limitations.md
├── .github/
|   └── workflows/
|       └── ci.yml
├── docker-compose.yml
├── start.sh
├── AGENTS.md
├── README.md
├── LICENSE
└── .gitignore
```

Required tests

Data tests

```
valid import passes
missing column fails
invalid date fails
invalid duration fails
unknown corridor fails
unknown asset fails
unknown resource fails
unknown dependency fails
duplicate import is detected
repeated import is idempotent
```

Scheduling tests

```
train overlap is rejected
buffer overlap is rejected
exact boundary behaves consistently
different lines are handled correctly
different corridors are handled correctly
power mismatch prevents grouping
signalling mismatch prevents grouping
resource overlap is rejected
duration overflow is rejected
deadline violation is rejected
```

```
dependency violation is rejected
goods forecast risk is applied
```

Plan tests

```
invalid solver output is rejected
fallback output is validated
draft can be edited
approved plan is immutable
revision creates a new version
stale revision returns 409
approval requires authorized user
rejection requires reason
department view excludes drafts
department view shows integrated context
completed block survives replan
execution record survives replan
```

End-to-end test

```
import
→ validate
→ prioritize
→ generate windows
→ generate baseline
→ generate optimized plan
→ validate plan
→ edit draft
→ approve
→ department visibility
→ execute
→ metrics
→ emergency replan
→ revision approval
→ export
```

Required final prompt for OpenChamber

Use this prompt to bring your current repository into alignment:

```
You are the lead engineer performing the final requirements-alignment implementation

The project is a human-approved, explainable hybrid AI decision-support prototype for

Read these files first:
- AGENTS.md
- docs/implementation-spec.md
- docs/problem-understanding.md
- docs/architecture.md
- docs/data-dictionary.md
- docs/demo-script.md
```


- docs/manual-acceptance-execution.md
- docs/final-release-audit.md, if present

The expected business problem is:

Railway maintenance for Engineering, Traction Distribution, and Signal & Telecommunications

Current implementation must be treated as untrusted until verified.

Do not rewrite the repository.

Do not remove working functionality.

Do not invent railway operating rules.

Do not claim live railway integration.

Do not claim a trained ML model unless historical data is actually used and evaluated.

Do not allow invalid plans to be saved or approved.

Perform the work in phases, committing each phase separately.

PHASE 1 – BASELINE

- Run backend tests.
- Run frontend lint, type-check, and build.
- Run local startup and health checks.
- Verify clean seed/reset.
- Verify Docker configuration.
- Save results to docs/baseline-report.md.
- Fix only setup or reproducibility failures.

PHASE 2 – EXECUTION

- Verify POST /api/blocks/{block_id}/execution.
- Ensure BLK IDs are canonical.
- Ensure WND IDs cannot be ambiguously treated as blocks.
- Return structured 4xx errors for invalid requests.
- Ensure duplicate execution is handled.
- Add API tests and frontend error handling.
- Preserve execution history during replanning.

PHASE 3 – DEPARTMENT VISIBILITY

Implement prototype department context:

- CONTROL_OFFICE;
- ENGINEERING;
- S_AND_T;
- TRACTION;
- PROJECTS;
- VIEWER;
- ADMIN.

Add:

- GET /api/approved-plans?department=...
- GET /api/plans/{plan_id}/department-view?department=...
- GET /api/notifications?department=...
- department selector;
- approved-plan calendar;
- my department tasks;
- integrated cross-department context;
- change notifications.

Draft plans must not appear in approved views.

PHASE 4 – CONTROLLED EDITING

- Draft plans can be edited.
- Approved and published plans are immutable.
- Approved-plan changes create a new revision.
- Revisions have `base_plan_id` and `revision_number`.
- Every change records old value, new value, editor, time, and reason.
- Every revision is revalidated before approval.
- Stale revisions return 409.
- Completed and approved work cannot be moved.
- Add edit, revision, history, and validation endpoints.
- Add frontend change summary and validation panel.

PHASE 5 – HYBRID AI TRANSPARENCY

Keep the current explainable hybrid design:

- feature extraction;
- normalized priority score;
- goods-risk analysis;
- compatibility reasoning;
- candidate-window ranking;
- baseline FCFS;
- CP-SAT optimization;
- deterministic fallback;
- independent validator.

Expose:

- score;
- rank;
- breakdown;
- reason;
- rule version;
- solver status;
- runtime;
- objective components;
- baseline metrics;
- optimized metrics;
- unscheduled reasons.

Do not use an opaque ML model without sufficient historical data.

PHASE 6 – REQUIREMENTS VERIFICATION

Verify:

- TMS integration adapter;
- SMMS integration adapter;
- TDMS integration adapter;
- COA integration adapter;
- timetable integration;
- goods forecast;
- resource availability;
- weekly plan;
- monthly plan;
- asset downtime objective;
- asset availability metrics;
- multi-department grouping;
- train and goods conflict checking;

- human approval;
- department visibility;
- editing and versioning;
- execution feedback;
- re-optimization.

PHASE 7 – FINAL TEST

Run:

- complete backend test suite;
- frontend lint;
- frontend type-check;
- frontend build;
- clean seed;
- weekly generation;
- monthly generation;
- approval;
- rejection;
- department view;
- draft editing;
- approved revision;
- valid execution;
- invalid execution;
- duplicate execution;
- emergency replan;
- metrics;
- export;
- Docker Compose validation.

Save:

- docs/final-acceptance-report.md;
- docs/demo-evidence.md.

For every failure, report:

- exact command;
- expected result;
- actual result;
- root cause;
- severity;
- file;
- proposed fix.

Do not declare completion unless the complete end-to-end flow works without HTTP 500

Final implementation principle

The final system should follow this immutable rule:

No data → no plan.
Invalid data → no planning.
Infeasible window → no assignment.
Invalid solver result → no draft.
Unvalidated draft → no approval.
Unapproved plan → no publication.
Completed work → no silent movement.

Approved work → revision required.
Every change → audit record.

That rule makes the architecture unambiguous, explainable, testable, and aligned with the required integrated block-planning problem.



1. [https://indianrailways.gov.in/railwayboard/uploads/directorate/tracks/Track_2/2024/Joint Procedure Order.pdf](https://indianrailways.gov.in/railwayboard/uploads/directorate/tracks/Track_2/2024/Joint_Procedure_Order.pdf)
2. <https://www.scribd.com/document/945768885/Rolling-block>
3. <https://www.mdpi.com/2076-3417/13/18/10503/pdf?version=1695219310>
4. <https://github.com/google/or-tools/blob/main/ortools/sat/docs/scheduling.md>
5. <https://github.com/google/or-tools/blob/stable/ortools/sat/docs/scheduling.md>
6. <https://deepwiki.com/google/or-tools/4.1.3-scheduling-constraints>
7. <https://atna-mam.utcluj.ro/index.php/Acta/article/view/2611>
8. <https://www.scribd.com/document/789144888/Rolling-Block>
9. <https://www.sciencedirect.com/science/article/abs/pii/S0968090X16302340>
10. [https://iricen.gov.in/iricen/ipwe_seminar/2017/Nov_2023_Vol-2/Ensuring Effectiveness in the Implementation of the Rolling Blockplan.pdf](https://iricen.gov.in/iricen/ipwe_seminar/2017/Nov_2023_Vol-2/Ensuring_Effectiveness_in_the_Implementation_of_the_Rolling_Blockplan.pdf)
11. <https://drops.dagstuhl.de/storage/01oasics/oasics-vol065-atmos2018/OASlcs.ATMOS.2018.1/OASlcs.ATMOS.2018.1.pdf>
12. <https://railnet.in/sr/mdzti/content/files/Chapter-15.pdf>
13. <https://www.scribd.com/presentation/694376913/New-Introduction-to-Rolling-Block-Programme-1>
14. <https://journals.sagepub.com/doi/pdf/10.1177/16878132221117124>
15. <https://www.mdpi.com/2076-3417/10/17/6016/pdf>
16. <https://downloads.hindawi.com/journals/jat/2023/3196066.pdf>
17. [https://figshare.com/articles/journal_contribution/Simultaneous Schedule of Trains and Track Maintenance According to Stochastic Blockage Time/19131644/1/files/33993470.pdf](https://figshare.com/articles/journal_contribution/Simultaneous_Schedule_of_Trains_and_Track_Maintenance_According_to_Stochastic_Blockage_Time/19131644/1/files/33993470.pdf)